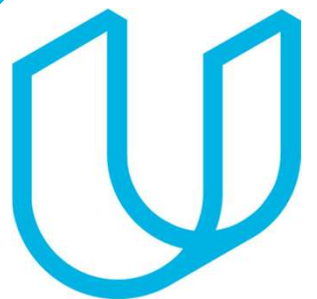


Tech ABC Corp - HR Database

[Arjun Avalakki Mohan & 25/05/2026]



Business Scenario

Business requirement

Tech ABC Corp saw explosive growth with a sudden appearance onto the gaming scene with their new AI-powered video game console. As a result, they have gone from a small 10 person operation to 200 employees and 5 locations in under a year. HR is having trouble keeping up with the growth, since they are still maintaining employee information in a spreadsheet. While that worked for ten employees, it has become increasingly cumbersome to manage as the company expands.

As such, the HR department has tasked you, as the new data architect, to design and build a database capable of managing their employee information.

Dataset

The [HR dataset](#) you will be working with is an Excel workbook which consists of 206 records, with eleven columns. The data is in human readable format, and has not been normalized at all. The data lists the names of employees at Tech ABC Corp as well as information such as job title, department, manager's name, hire date, start date, end date, work location, and salary.

IT Department Best Practices

The IT Department has certain Best Practices policies for databases you should follow, as detailed in the [Best Practices document](#).



Step 1

Data Architecture Foundations

Step 1: Data Architecture Foundations

Hi,

Welcome to Tech ABC Corp. We are excited to have some new talent onboard. As you may already know, Tech ABC Corp has recently experienced a lot of growth. Our AI powered video game console WOPR has been hugely successful and as a result, our company has grown from 10 employees to 200 in only 6 months (and we are projecting a 20% growth a year for the next 5 years). We have also grown from our Dallas, Texas office, to 4 other locations nationwide: New York City, NY, San Francisco, CA, Minneapolis, MN, and Nashville, TN.

While this growth is great, it is really starting to put a strain on our record keeping in HR. We currently maintain all employee information on a shared spreadsheet. When HR consisted of only myself, managing everyone on an Excel spreadsheet was simple, but now that it is a shared document I am having serious reservations about data integrity and data security. If the wrong person got their hands on the HR file, they would see the salaries of every employee in the company, all the way up to the president.

After speaking with Jacob Lauber, the manager of IT, he suggested I put in a request to have my HR Excel file converted into a database. He suggested I reach out to you as I am told you have experience in designing and building databases. When you are building this, please keep in mind that I want any employee with a domain login to be have read only access the database. I just don't want them having access to salary information. That needs to be restricted to HR and management level employees only. Management and HR employees should also be the only ones with write access. By our current estimates, 90% of users will be read only.

I also want to make sure you know that am looking to turn my spreadsheet into a live database, one I can input and edit information into. I am not really concerned with reporting capabilities at the moment. Since we are working with employee data we are required by federal regulations to maintain this data for at least 7 years; additionally, since this is considered business critical data, we need to make sure it gets backed up properly.

As a final consideration. We would like to be able to connect with the payroll department's system in the future. They maintain employee attendance and paid time off information. It would be nice if the two systems could interface in the future

I am looking forward to working with you and seeing what kind of database you design for us.

Thanks,
Sarah Collins
Head of HR

Data Architect Business Requirement

- **Purpose of the new database:**

The purpose of the new database is to provide HR with a secure and reliable system to manage employee information as the company continues to grow. It is intended to replace the current shared spreadsheet, improve data integrity and security, and provide a live system where employee information can be updated and maintained more efficiently.

- **Describe current data management solution:**

The current HR solution is based on a shared spreadsheet.

- Employee data is stored in Excel.
- The spreadsheet is manually maintained by HR.
- The solution is becoming difficult to manage as the company grows.
- There are concerns around data security and data integrity.

- **Describe current data available:**

The current spreadsheet contains employee information including:

- | | |
|-----------------|-------------------------|
| - Employee ID | - Manager name |
| - Employee name | - Salary |
| - Email address | - Start and end dates |
| - Hire date | - Work location details |
| - Job title | - Education level |
| - Department | |

Data Architect Business Requirement

- **Additional data requests:**

The HR department would like the database to support future integration with the payroll system. Future integration may include:

- Employee attendance information
- Paid time off (PTO) information
- Payroll-related employee data

- **Who will own/manage data**

The HR department will own and manage the employee data stored in the database. Responsibilities includes:

- Maintaining employee records
- Managing updates and corrections
- Controlling access to sensitive information
- Ensuring employee information remains accurate and up to date

- **Who will have access to database**

The database will be accessible to employees with a company domain login. Access restrictions will include:

- General employees will have read-only access
- HR and management employees will have write access
- Salary information will be restricted to HR and management users only

Data Architect Business Requirement

- **Estimated size of database**

The current database is expected to contain approximately:

- 205 employee records (rows)
- 15 data fields (columns)

- **Estimated annual growth**

The business is projecting approximately:

- 20% annual growth in employee data
- Continued expansion across multiple office locations over the next 5 years

- **Is any of the data sensitive/restricted**

The following information should be treated as sensitive and restricted:

- Employee salary information
- Employee personal information
- HR management and reporting structure information

Data Architect Technical Requirement

- **Justification for the new database**

The new database is required to:

- Improve data security and restrict access to sensitive salary information
- Improve data integrity and reduce risks caused by manual spreadsheet updates
- Support future business growth and increasing employee records
- Provide a centralized system for managing employee information

- **Database objects**

The database will include the following objects:

- Employee table
- Department table
- Job title table
- Location table
- Education level table
- Employee job history table
- Employee salary table

- **Data ingestion**

The proposed data ingestion method is “ETL (Extract, Transform, Load)”. This approach is suitable because the current employee data is stored in Excel spreadsheets and will need to be cleaned and loaded into the new database.

Data Architect Technical Requirement

- **Data governance (Ownership and User access)**

Ownership: HR will own and maintain the employee data. IT will manage database security, backups, and access control.

User Access: General employees will have read-only access. HR and management users will have read and write access. Salary information will only be accessible to HR and management users.

- **Scalability**

Replication or sharding is not required at this stage. The database size is small, and expected growth can be handled by a standard relational database design.

- **Flexibility**

The database will use unique employee IDs to support future system integration. Data will be stored in normalized tables to support future payroll integration.

Storage & retention

Storage (disk or in-memory): Disk-based storage will be used for the database.

Retention: Employee data must be retained for a minimum of 7 years.

- **Backup**

- Weekly full and Daily incremental backups will be performed.
- Backups are required because the database contains business critical HR data



Step 2

Relational Database Design

Step 2: Relational Database Design

This step is where you will go through the process of designing a new database for Tech ABC Corp's HR department. Using the [dataset](#) provided, along with the requirements gathered in step one, you are going to develop a relational database set to the 3NF.

Using Lucidchart, you will create 3 entity relationship diagrams (ERDs) to show how you developed the final design for your data.

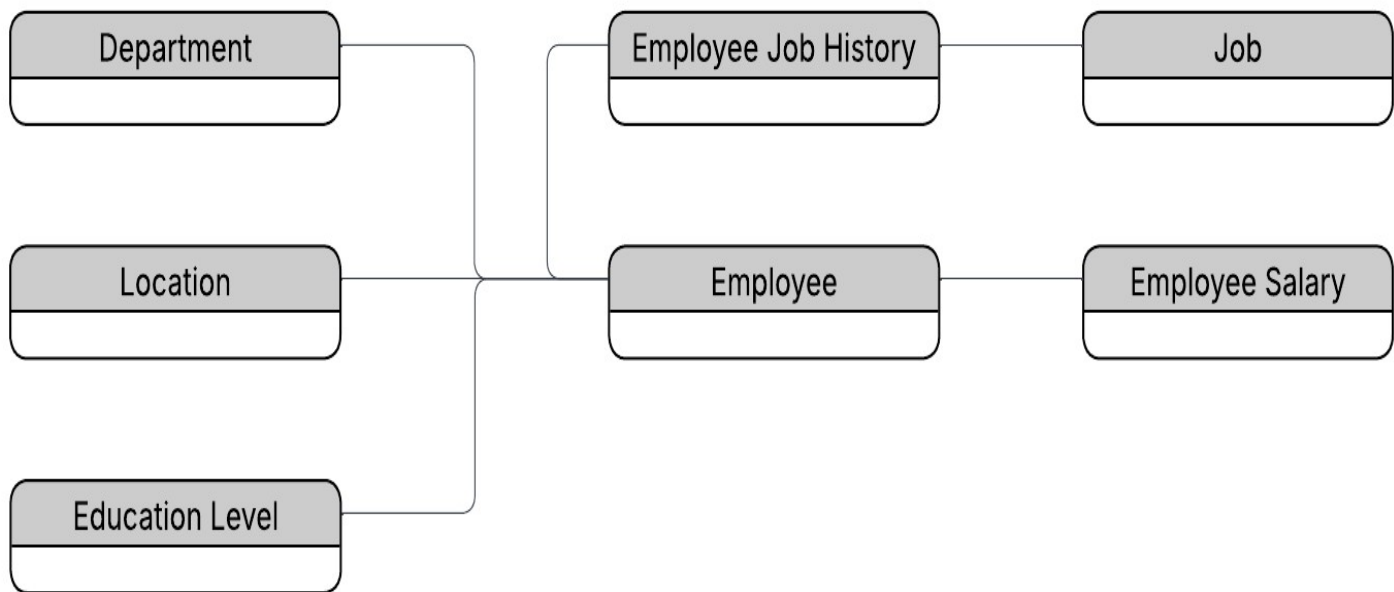
You will submit a screenshot for each of the 3 ERDs you create. You will find detailed instructions for developing each of the ERDs over the next several pages.

ERD

- **Conceptual**

This is the most general level of data modeling. At the conceptual level, you should be thinking about creating entities that represent business objects for the database. Think broadly here. Attributes (or column names) are not required at this point, but relationship lines are required (although Crow's foot notation is not needed at this level). Create at least three entities for this model; thinking about the 3NF will aid you in deciding the type of entities to create.

Use Lucidchart's built-in template for DBMS ER Diagram UML.

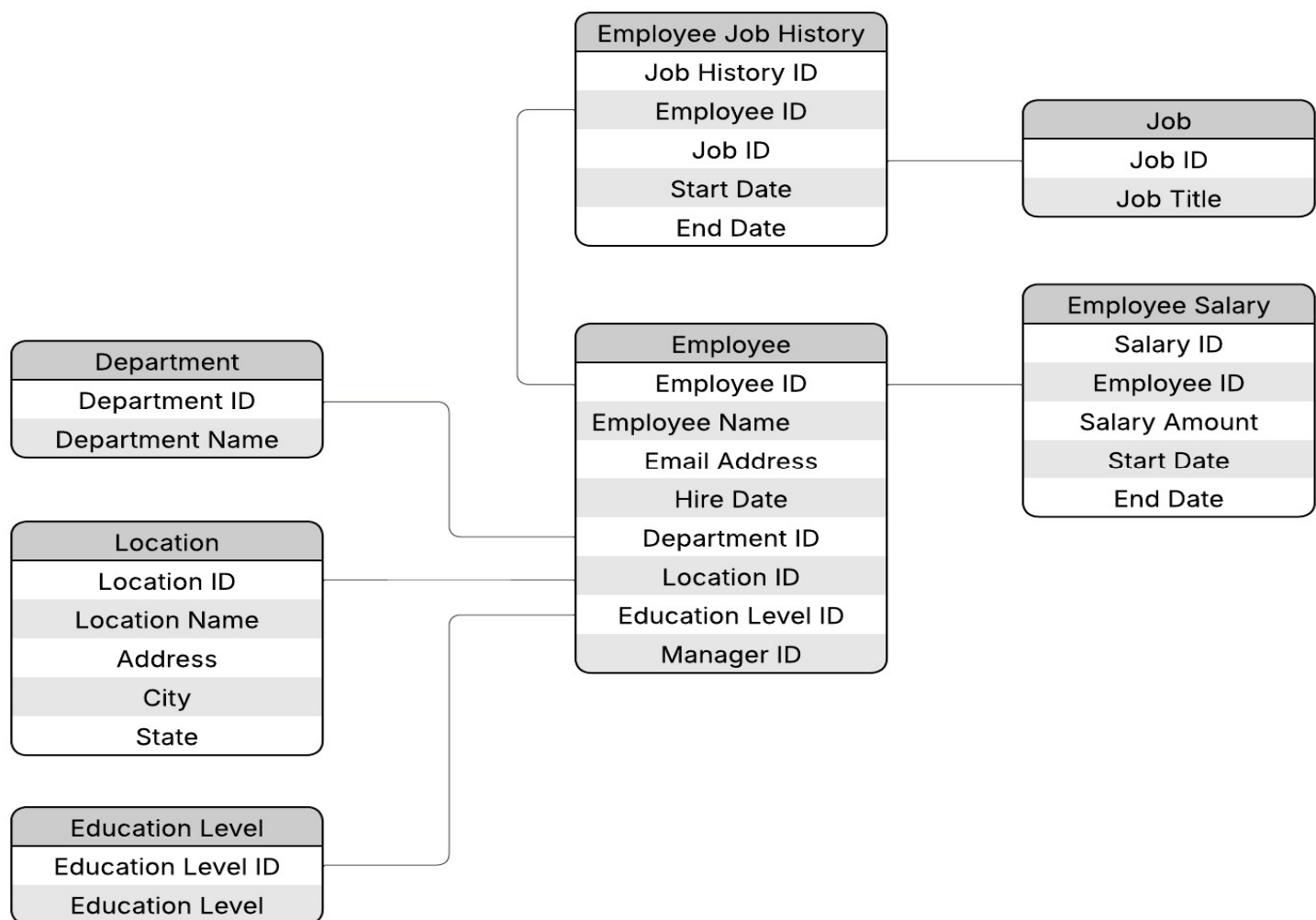


ERD

- **Logical**

The logical model is the next level of refinement from the conceptual ERD. At this point, you should have normalized the data to the 3NF. Attributes should also be listed now in the ERD. You can still use human-friendly entity and attribute names in the logical model, and while relationship lines are required, Crow's foot notation is still not needed at this point.

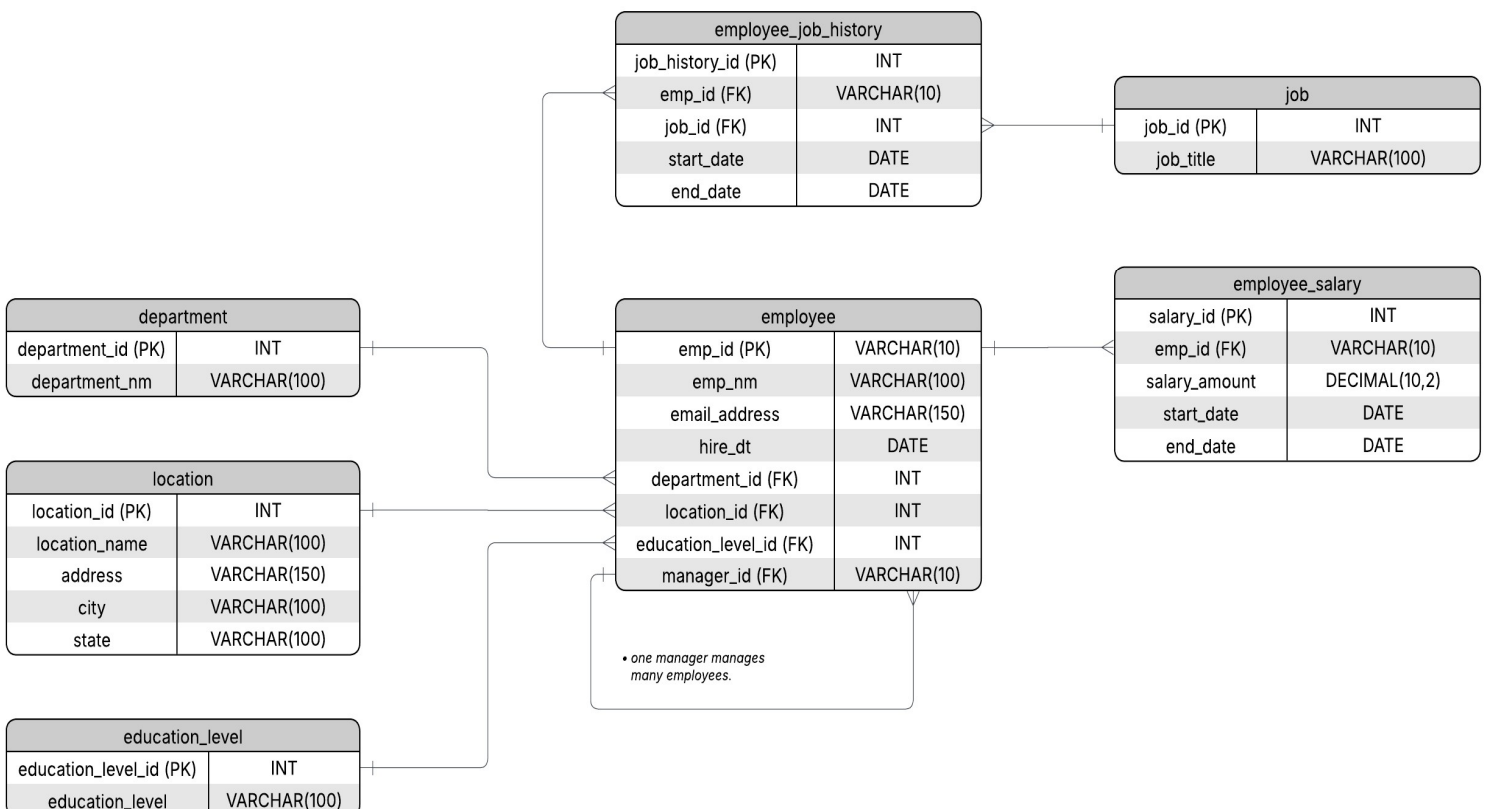
Use Lucidchart's built-in template for DBMS ER Diagram UML.



ERD

- Physical

The physical model is what will be built in the database. Each entity should represent a database table, complete with column names and data types. Primary keys and foreign keys should also be represented here. Primary keys should be in bold type with the (PK) designation following the field name. Foreign keys should be in normal type face, but have the designation (FK) after the column name. Finally, in the physical model, Crow's foot notation is important.





Step 3

Create A Physical
Database

Step 3: Create A Physical Database

In this step, you will be turning your database model into a physical database.

You will:

- Create the database using SQL DDL commands
- Load the data into your database, utilizing flat file ETL
- Answer a series of questions using CRUD SQL commands to demonstrate your database was created and populated correctly

Submission

For this step, you will need to submit SQL files containing all DDL SQL scripts used to create the database.

You will also have to submit screenshots showing CRUD commands, along with results for each of the questions found in the starter template.

Hints

Your DDL script will be graded by running the code you submit. Please ensure your SQL code runs properly!

Foreign keys cannot be created on tables that do not exist yet, so it may be easier to create all tables in the database, then to go back and run modify statements on the tables to create foreign key constraints.

After running CRUD commands like update, insert, or delete, run a `SELECT*` command on the affected table, so the reviewer can see the results of the command.

DDL

Create a DDL SQL script capable of building the database you designed in Step 2

01_create_tables.sql

```
CREATE TABLE department (  
    department_id INT PRIMARY KEY,  
    department_nm VARCHAR(100)  
);  
  
CREATE TABLE location (  
    location_id INT PRIMARY KEY,  
    location_name VARCHAR(100),  
    address VARCHAR(150),  
    city VARCHAR(100),  
    state VARCHAR(100)  
);  
  
CREATE TABLE education_level (  
    education_level_id INT PRIMARY KEY,  
    education_level VARCHAR(100)  
);  
  
CREATE TABLE job (  
    job_id INT PRIMARY KEY,  
    job_title VARCHAR(100)  
);  
  
CREATE TABLE employee (  
    emp_id VARCHAR(10) PRIMARY KEY,  
    emp_nm VARCHAR(100),  
    email_address VARCHAR(150),  
    hire_dt DATE,  
    department_id INT,  
    location_id INT,  
    education_level_id INT,  
    manager_id VARCHAR(10)  
);  
  
CREATE TABLE employee_salary (  
    salary_id INT PRIMARY KEY,  
    emp_id VARCHAR(10),  
    salary_amount DECIMAL(10,2),  
    start_date DATE,  
    end_date DATE  
);  
  
CREATE TABLE employee_job_history (  
    job_history_id INT PRIMARY KEY,  
    emp_id VARCHAR(10),  
    job_id INT,  
    start_date DATE,  
    end_date DATE  
);
```

```
ALTER TABLE employee  
ADD CONSTRAINT fk_employee_department  
FOREIGN KEY (department_id)  
REFERENCES department(department_id);  
  
ALTER TABLE employee  
ADD CONSTRAINT fk_employee_location  
FOREIGN KEY (location_id)  
REFERENCES location(location_id);  
  
ALTER TABLE employee  
ADD CONSTRAINT fk_employee_education  
FOREIGN KEY (education_level_id)  
REFERENCES education_level(education_level_id);  
  
ALTER TABLE employee  
ADD CONSTRAINT fk_employee_manager  
FOREIGN KEY (manager_id)  
REFERENCES employee(emp_id);  
  
ALTER TABLE employee_salary  
ADD CONSTRAINT fk_salary_employee  
FOREIGN KEY (emp_id)  
REFERENCES employee(emp_id);  
  
ALTER TABLE employee_job_history  
ADD CONSTRAINT fk_job_history_employee  
FOREIGN KEY (emp_id)  
REFERENCES employee(emp_id);  
  
ALTER TABLE employee_job_history  
ADD CONSTRAINT fk_job_history_job  
FOREIGN KEY (job_id)  
REFERENCES job(job_id);
```

DDL

Create a DDL SQL script capable of building the database you designed in Step 2

02_etl_load_data.sql

```
-- Load department lookup table
INSERT INTO department (department_id, department_nm)
SELECT ROW_NUMBER() OVER (ORDER BY department_nm), department_nm FROM (SELECT DISTINCT department_nm FROM proj_stg) d;

-- Load location lookup table
INSERT INTO location (location_id, location_name, address, city, state)
SELECT ROW_NUMBER() OVER (ORDER BY location, address, city, state), location, address, city, state FROM (SELECT DISTINCT location, address, city, state FROM proj_stg) l;

-- Load education level lookup table
INSERT INTO education_level (education_level_id, education_level)
SELECT ROW_NUMBER() OVER (ORDER BY education_lvl), education_lvl FROM (SELECT DISTINCT education_lvl FROM proj_stg) e;

-- Load job lookup table
INSERT INTO job (job_id, job_title)
SELECT ROW_NUMBER() OVER (ORDER BY job_title), job_title FROM (SELECT DISTINCT job_title FROM proj_stg) j;

-- Load employee table
INSERT INTO employee (emp_id, emp_nm, email_address, hire_dt, department_id, location_id, education_level_id, manager_id)
SELECT DISTINCT ON (s.emp_id) s.emp_id, s.emp_nm, s.email, s.hire_dt, d.department_id, l.location_id, e.education_level_id, m.emp_id AS manager_id
FROM proj_stg s JOIN department d ON s.department_nm = d.department_nm JOIN location l ON s.location = l.location_name AND s.address = l.address AND s.city = l.city
AND s.state = l.state JOIN education_level e ON s.education_lvl = e.education_level LEFT JOIN (SELECT DISTINCT emp_id, emp_nm FROM proj_stg) m ON s.manager = m.emp_nm
ORDER BY s.emp_id, s.start_dt DESC;

-- Load employee salary table
INSERT INTO employee_salary (salary_id, emp_id, salary_amount, start_date, end_date)
SELECT ROW_NUMBER() OVER (ORDER BY emp_id, start_dt), emp_id, salary, start_dt, end_dt FROM proj_stg;

-- Load employee job history table
INSERT INTO employee_job_history (job_history_id, emp_id, job_id, start_date, end_date)
SELECT ROW_NUMBER() OVER (ORDER BY s.emp_id, s.start_dt), s.emp_id, j.job_id, s.start_dt, s.end_dt FROM proj_stg s JOIN job j ON s.job_title = j.job_title;

-- Validation checks
SELECT * FROM department;
SELECT * FROM location;
SELECT * FROM education_level;
SELECT * FROM job;
SELECT * FROM employee LIMIT 10;
SELECT * FROM employee_salary LIMIT 10;
SELECT * FROM employee_job_history LIMIT 10;
```

CRUD

- Question 1: Return a list of employees with Job Titles and Department Names

scratchpad.sql

StageTableLoad.sql

```
1 -- Question 1
2 SELECT e.emp_id, e.emp_nm, j.job_title, d.department_nm FROM employee e JOIN employee_job_history ejh ON e.emp_id = ejh.emp_id JOIN job j
   ON ejh.job_id = j.job_id JOIN department d ON e.department_id = d.department_id ORDER BY e.emp_nm;
3
```

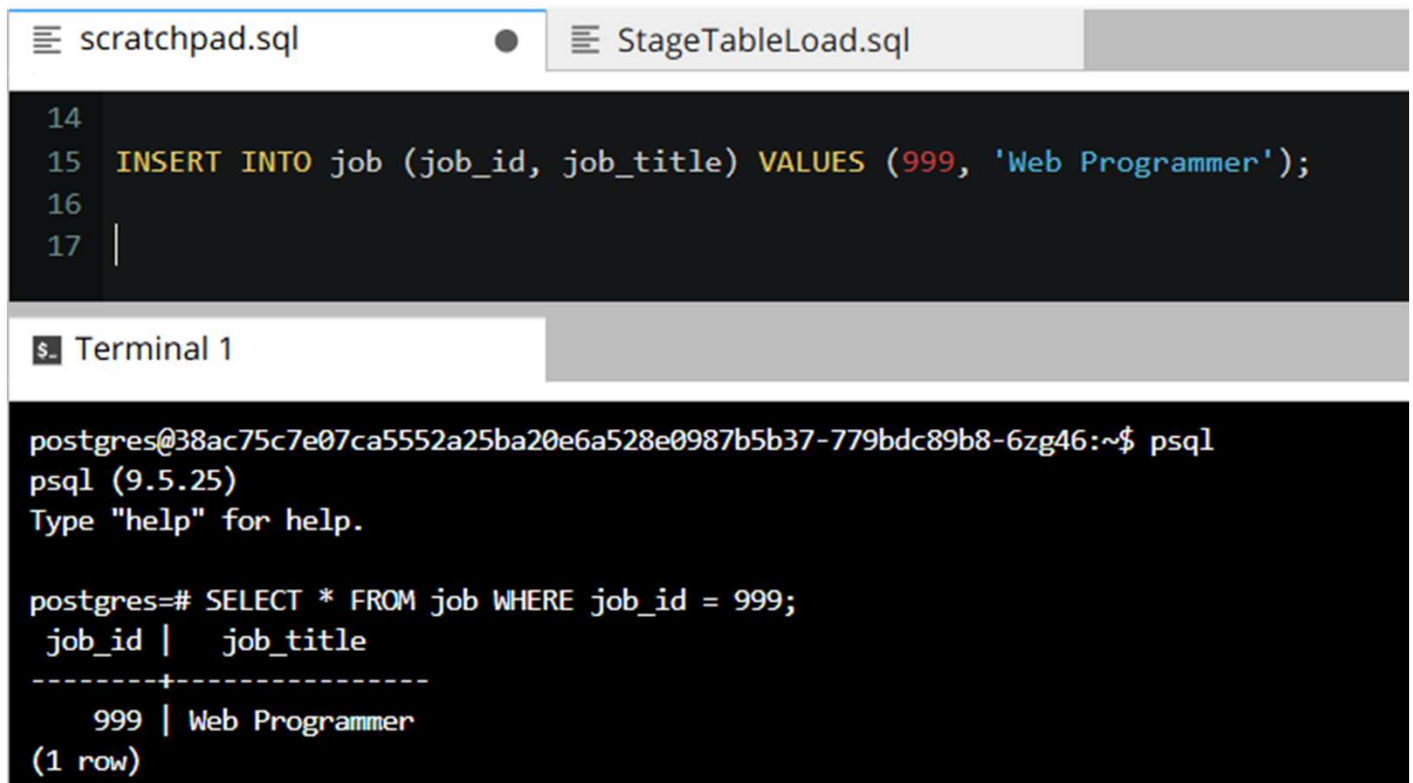
Terminal 1

emp_id	emp_nm	job_title	department_nm
E14913	Aaron Gordon	Network Engineer	Product Development
E91791	Aaron Richman	Administrative Assistant	Product Development
E16678	Abby Lockhart	Network Engineer	IT
E16678	Abby Lockhart	Database Administrator	IT
E37523	Alan Mecklet	Administrative Assistant	Product Development
E74490	Alejandro Scannapieco	Sales Rep	Sales
E55880	Alex Warring	Shipping and Receiving	Distribution
E48148	Alexis Fitzpatrick	Administrative Assistant	IT
E63041	Allison Gentle	Manager	Distribution
E89540	Amit Hardiya	Administrative Assistant	Sales
E16276	Analyn Braza	Sales Rep	Sales
E23429	Andrew Yoon	Sales Rep	Product Development
E79552	Angela Masdary	Network Engineer	IT
E87822	Anil Padala	Software Engineer	Product Development
E18697	Anita Deluise	Administrative Assistant	HQ
E50012	Ann Roberto	Administrative Assistant	IT
E79464	Anu Patel	Legal Counsel	HQ
E40432	April Briggs	Software Engineer	IT
E18659	Arnold Hanson	Software Engineer	Product Development
E82137	Arup Das	Software Engineer	IT
E35053	Ashley Bergman	Administrative Assistant	Distribution
E62527	Barry Walsh	Network Engineer	IT
E37389	Becky Weaver	Sales Rep	Sales
E29129	Belinda Van Loon	Sales Rep	Sales
E43694	Benyam Gizaw	Legal Counsel	Sales
E30058	Bertin Dakouo	Sales Rep	Product Development
E35856	Beth Sepkowski	Software Engineer	IT
E25710	Brent Robinson	Design Engineer	Product Development
E95199	Carlos Fernandes	Sales Rep	Product Development

--More--

CRUD

- Question 2: Insert Web Programmer as a new job title



The screenshot shows a code editor with two tabs: 'scratchpad.sql' and 'StageTableLoad.sql'. The 'scratchpad.sql' tab is active and contains the following SQL code:

```
14
15 INSERT INTO job (job_id, job_title) VALUES (999, 'Web Programmer');
16
17 |
```

Below the code editor is a terminal window titled 'Terminal 1'. It shows the execution of the SQL command using 'psql' and the resulting output:

```
postgres@38ac75c7e07ca5552a25ba20e6a528e0987b5b37-779bdc89b8-6zg46:~$ psql
psql (9.5.25)
Type "help" for help.

postgres=# SELECT * FROM job WHERE job_id = 999;
 job_id |  job_title
-----+-----
    999 | Web Programmer
(1 row)
```

CRUD

- Question 3: Correct the job title from web programmer to web developer

scratchpad.sql

StageTableLoad.sql

```
14
15 -- INSERT INTO job (job_id, job_title) VALUES (999, 'Web Programmer');
16
17 UPDATE job SET job_title = 'Web Developer' WHERE job_id = 999;
```

Terminal 1

```
postgres@38ac75c7e07ca5552a25ba20e6a528e0987b5b37-779bdc89b8-6zg46:~$ psql
psql (9.5.25)
Type "help" for help.
```

```
postgres=# SELECT * FROM job WHERE job_id = 999;
 job_id |  job_title
-----+-----
    999 | Web Programmer
(1 row)
```

```
postgres=# \i /home/workspace/scratchpad.sql
UPDATE 1
postgres=# SELECT * FROM job WHERE job_id = 999;
 job_id |  job_title
-----+-----
    999 | Web Developer
(1 row)
```

CRUD

- Question 4: Delete the job title Web Developer from the database

scratchpad.sql

StageTableLoad.sql

```
14
15 -- INSERT INTO job (job_id, job_title) VALUES (999, 'Web Programmer');
16
17 -- UPDATE job SET job_title = 'Web Developer' WHERE job_id = 999;
18
19 DELETE FROM job WHERE job_id = 999;
```

\$ Terminal 1

```
postgres@38ac75c7e07ca5552a25ba20e6a528e0987b5b37-779bdc89b8-6zg46:~$ psql
psql (9.5.25)
Type "help" for help.
```

```
postgres=# SELECT * FROM job WHERE job_id = 999;
 job_id |  job_title
-----+-----
    999 | Web Programmer
(1 row)
```

```
postgres=# \i /home/workspace/scratchpad.sql
UPDATE 1
postgres=# SELECT * FROM job WHERE job_id = 999;
 job_id |  job_title
-----+-----
    999 | Web Developer
(1 row)
```

```
postgres=# \i /home/workspace/scratchpad.sql
DELETE 1
postgres=# SELECT * FROM job WHERE job_id = 999;
 job_id |  job_title
-----+-----
(0 rows)
```

CRUD

- Question 5: How many employees are in each department?

scratchpad.sql

StageTableLoad.sql

```
20
21 SELECT d.department_nm, COUNT(e.emp_id) AS total_employees FROM department d
22 JOIN employee e ON d.department_id = e.department_id GROUP BY d.department_nm ORDER BY total_employees DESC;
23
```

Terminal 1

```
postgres@38ac75c7e07ca5552a25ba20e6a528e0987b5b37-779bdc89b8-6zg46:~$ psql
```

```
psql (9.5.25)
```

```
Type "help" for help.
```

```
postgres=# \i /home/workspace/scratchpad.sql
```

department_nm	total_employees
Product Development	69
IT	52
Sales	40
Distribution	25
HQ	13

(5 rows)

CRUD

- Question 6: Write a query that returns current and past jobs (include employee name, job title, department, manager name, start and end date for position) for employee Toni Lembeck.

scratchpad.sql

StageTableLoad.sql

```
23
24 SELECT e.emp_nm AS employee_name, j.job_title, d.department_nm, m.emp_nm AS manager_name, ejh.start_date, ejh.end_date FROM employee e
25 JOIN employee_job_history ejh ON e.emp_id = ejh.emp_id JOIN job j ON ejh.job_id = j.job_id JOIN department d ON e.department_id =
  d.department_id LEFT JOIN employee m ON e.manager_id = m.emp_id WHERE e.emp_nm = 'Toni Lembeck';
```

Terminal 1

```
postgres@38ac75c7e07ca5552a25ba20e6a528e0987b5b37-779bdc89b8-6zg46:~$ psql
```

```
psql (9.5.25)
```

```
Type "help" for help.
```

```
postgres=# \i /home/workspace/scratchpad.sql
```

employee_name	job_title	department_nm	manager_name	start_date	end_date
Toni Lembeck	Database Administrator	IT	Jacob Lauber	2001-07-18	2100-02-02
Toni Lembeck	Network Engineer	IT	Jacob Lauber	1995-03-12	2001-07-18

(2 rows)

CRUD

- **Question 7: Describe how you would apply table security to restrict access to employee salaries using an SQL server.**

I would apply table-level security by creating separate database roles for normal users and HR or management users. Normal users should only have read-only access to non-sensitive tables such as employee, department, job, location, employee_job_history, and education_level. They should not have access to the employee_salary table.

Example SQL:

```
CREATE ROLE readonly_user;  
GRANT SELECT ON employee, department, job, location, employee_job_history,  
education_level TO readonly_user;  
REVOKE ALL ON employee_salary FROM readonly_user;
```

For HR and management users, I would create a separate role with access to salary information.

Example SQL:

```
CREATE ROLE hr_manager;  
GRANT SELECT, INSERT, UPDATE ON employee_salary TO hr_manager;  
GRANT SELECT, INSERT, UPDATE ON employee, department, job, location,  
employee_job_history, education_level TO hr_manager;
```

This approach ensures that only authorized users can access or update sensitive employee salary information.